VTI MARKETPLACE LMS

Master Product & Development Specification

Udemy-Class Learning Marketplace + Enterprise LMS + VTI-Specific Operations

---

0. ROLE OF THE AI DEVELOPMENT AGENT

You are the **Lead Product Architect, Senior Full-Stack Engineer, UI/UX Engineer, Security Engineer, QA Engineer and Technical Product Manager** for this project.

The objective is NOT merely to build a basic LMS.

The objective is to build a **commercial-grade, reusable, multi-tenant-capable online learning marketplace platform** comparable in feature depth to modern platforms such as Udemy, while preserving all VTI-specific requirements.

This platform must be designed so that:

1. It can serve the current VTI client.
2. It can later be rebranded and sold to other education businesses.
3. It can support different organizations, instructors and course catalogs.
4. It can eventually support a native mobile application.
5. Its business logic is not tightly coupled to VTI branding.
6. New payment providers, email providers, storage providers and messaging providers can be added without rewriting the core application.
7. The architecture remains maintainable as the product grows.

Think of this project as:

"Build a reusable Udemy-class learning marketplace engine, then configure VTI on top of it."

Do NOT think of this as:

"Build one LMS specifically for VTI."

---

1. IMPORTANT: USE UDEMY AS A PRODUCT REFERENCE

You are explicitly authorized to research the publicly accessible Udemy website and official Udemy documentation as a **product and UX reference**.

Before implementing marketplace functionality, you may and should visit:

https://www.udemy.com/

and relevant official Udemy help/documentation pages.

Study the current publicly observable product as it exists **at the time of development**, rather than relying only on memory.

Inspect:

- Homepage
- Header/navigation
- Search
- Category navigation
- Course discovery
- Search results
- Filters
- Sorting
- Course cards
- Course detail pages
- Course landing pages
- Instructor profiles
- Cart
- Wishlist
- Checkout
- Coupons
- Student dashboard
- My Learning
- Course player

- Reviews
- Q&A
- Notes
- Resources
- Instructor dashboard
- Course creation flow
- Course management
- Instructor analytics
- Communications
- Certificates
- Promotions
- Subscription-related experiences
- Relevant account/settings pages
- Mobile responsive behavior

Also inspect official documentation to identify publicly documented features.

IMPORTANT INTELLECTUAL PROPERTY RULE

Do NOT copy:

- Udemy source code
- Udemy proprietary assets
- Udemy logos
- Udemy trademarks
- Udemy images
- copyrighted text
- proprietary design files
- proprietary backend implementation
- private APIs

- private data

Do NOT produce a pixel-for-pixel clone.

Instead:

Use Udemy as a **functional, information-architecture and UX benchmark**.

Recreate the **capabilities, workflows, product depth and interaction patterns** using an original VTI-compatible design system and original components.

The resulting product must have its own:

- branding
- typography
- colors
- spacing system
- icons
- illustrations
- imagery
- component styling
- copy
- visual identity

The goal is:

Udemy-level product capability — original product implementation.

---

2. FIRST TASK — PRODUCT DISCOVERY BEFORE CODING

Before changing or creating application code, inspect the existing repository.

Determine:

- Current framework version
- Existing folder structure
- Existing routes

- Existing database schema
- Existing Prisma models
- Existing authentication
- Existing roles
- Existing APIs
- Existing components
- Existing state management
- Existing course system
- Existing student dashboard
- Existing instructor system
- Existing admin system
- Existing payment implementation
- Existing storage implementation
- Existing tests
- Existing middleware/proxy/auth guards
- Existing environment variables
- Existing technical debt

DO NOT rebuild existing working functionality unnecessarily.

Create a short internal implementation assessment:

EXISTING

├— Complete

├— Partially complete

├— Broken

├— Missing

└— Needs architectural refactor

Then map the existing application against this specification.

The existing implementation is the source of truth for what has already been built.

---

3. PRODUCT VISION

The final product should contain these major layers:



Auth Payments Media

Email Webhooks CDN

Storage Notifications Search

Audit Jobs Analytics

---

4. USER TYPES

The system must support:

Platform-level

1. SUPER_ADMIN
2. ADMIN

Marketplace / learning

3. INSTRUCTOR
4. STUDENT

Architecture must allow future roles without rewriting authorization.

Use centralized permission checks rather than scattered:

if (user.role === "ADMIN")

throughout the application.

Prefer:

RBAC

+

Permission Service

+

Resource-level authorization

---

5. SUPER ADMIN

Super Admin has platform-wide authority.

Capabilities:

User management

- Create Admin
- Edit Admin
- Suspend Admin
- Reactivate Admin
- Delete Admin
- Create Instructor
- Edit Instructor
- Suspend Instructor
- Reactivate Instructor
- Delete/archive Instructor according to historical-data rules
- View Students
- Edit Students
- Suspend Students
- Manage student access

Only one protected initial Super Admin must exist unless the architecture explicitly supports multiple Super Admins later.

Platform management

Super Admin controls:

- Categories
- Subcategories
- Topics
- Courses
- Instructors

- Students
- Orders
- Payments
- Refunds
- Coupons
- Promotions
- Commissions
- Instructor payouts
- Certificates
- Reviews
- Reports
- Announcements
- CMS
- Notifications
- Email templates
- WhatsApp configuration
- Payment gateway
- Storage
- Platform settings
- Feature flags
- Audit logs
- Security settings

---

6. ADMIN

Admin capabilities must be permission-controlled.

Possible permissions include:

courses.view

courses.create

courses.edit

courses.publish

courses.unpublish

courses.moderate

students.view

students.edit

students.enroll

students.suspend

instructors.view

instructors.create

instructors.edit

instructors.assign

orders.view

payments.view

refunds.manage

reviews.moderate

support.manage

announcements.manage

reports.view

Super Admin can grant/revoke Admin permissions.

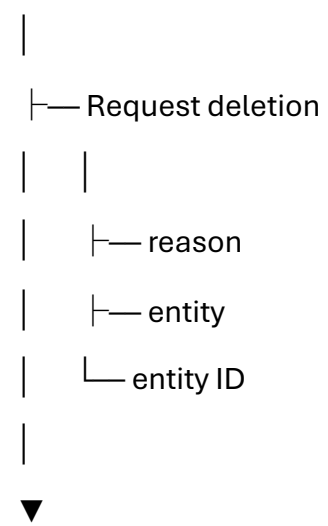
Admin must NOT automatically receive financial or destructive permissions.

---

7. ADMIN DELETION REQUEST WORKFLOW

Deleting important records must be a real workflow.

Example:



Store:

- requester
- entity
- reason
- requestedAt
- reviewer
- decision

- decisionReason
- decidedAt

Never implement this as merely a disabled delete button.

---

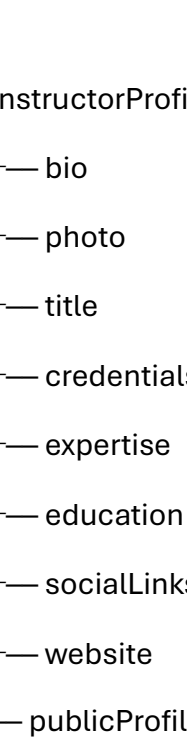
8. INSTRUCTOR MODEL

An Instructor is a real authenticated User.

Do NOT create a fake "instructor-only profile" disconnected from authentication.

Recommended conceptual model:

User



└── InstructorProfile

├── bio

├── photo

├── title

├── credentials

├── expertise

├── education

├── socialLinks

├── website

└── publicProfile

Instructor accounts:

- Cannot publicly self-register
- Are created by Admin/Super Admin
- Can log in
- Have their own dashboard
- Can manage assigned courses

- Can communicate with students
- Can answer Q&A
- Can view performance
- Can view student progress
- Can manage course content according to permissions
- Can be assigned as primary instructor or co-instructor

---

9. MARKETPLACE MODEL

This is the major expansion beyond the original VTI LMS.

The platform must behave like a **course marketplace**, not simply a private LMS.

A course should support:

Course

├— Basic Information

├— Landing Page

├— Pricing

├— Promotions

├— Curriculum

├— Instructors

├— Learning Objectives

├— Requirements

├— Intended Learners

├— Topic

├— Category

├— Subcategory

├— Language

├── Level

├── Captions

├── Preview Content

├── Reviews

├── Q&A

├── Resources

├── Course Messages

├── Students

├── Analytics

└── Publishing / Moderation

---

10. COURSE MARKETPLACE

Public visitors must be able to:

- Browse courses
- Search courses
- Filter courses
- Sort courses
- Open course landing pages
- Preview selected lectures
- View instructor profiles
- View ratings
- Read reviews
- View curriculum
- View learning objectives
- View requirements

- View intended learners
- View course duration
- View lecture count
- View language
- View subtitles/captions
- View last updated date
- View price
- Apply coupons
- Add to cart
- Add to wishlist
- Purchase
- Share course

---

11. COURSE DISCOVERY

Implement a robust search/discovery system.

Search should support:

- Course title
- Subtitle
- Description
- Instructor
- Category
- Subcategory
- Topic
- Skills
- Learning objectives

Filters should support where applicable:

- Category
- Subcategory
- Topic
- Price
- Rating
- Level
- Language
- Duration
- Features
- Captions
- Course type
- Instructor
- Newest
- Popular
- Relevance

Sorting:

Relevance

Highest Rated

Most Popular

Newest

Price Low → High

Price High → Low

The exact available filters should be configurable.

---

12. CATEGORY SYSTEM

Create a hierarchical taxonomy:

Category

└── Subcategory

    └── Topic

Example:

Development

├── Web Development

│   ├── JavaScript

│   ├── React

│   └── Next.js

└── Mobile Development

Do NOT hard-code categories.

Admins must manage the taxonomy.

---

13. COURSE LANDING PAGE

Create a premium marketplace course page.

Structure should contain:

Above the fold

- Breadcrumbs
- Course title
- Subtitle
- Rating
- Review count
- Student count
- Instructor
- Last updated
- Language

- Course level
- Hero/preview video
- Price
- Discount price
- Coupon/promotional information
- Add to cart
- Buy now
- Wishlist

Main content

- What you'll learn
- Course description
- Requirements
- Who this course is for
- Course curriculum
- Instructor section
- Student reviews
- Course Q&A
- Related courses
- Frequently bought together
- Recommended courses

Use the live Udemy experience as a reference for information architecture, but create an original visual implementation.

---

14. COURSE CURRICULUM

Support:

Course

├── Section
│ ├── Lecture
│ ├── Lecture
│ ├── Quiz
│ └── Resources
│
├── Section
│ ├── Lecture
│ ├── Assignment
│ └── Quiz

Lesson types should be extensible.

Initial types:

- VIDEO
- ARTICLE/TEXT
- PDF
- MIXED
- QUIZ
- ASSIGNMENT
- PRACTICE_TEST
- CODING_EXERCISE

Some advanced lesson types can initially be feature-flagged if implementation complexity is high.

---

15. VIDEO SYSTEM

Implement a proper media architecture.

Never store large video files directly in PostgreSQL.

Use:

Upload

↓

Object Storage

↓

Processing / Encoding

↓

CDN / Streaming

↓

Signed/authorized playback

Support:

- Video duration
- Thumbnail
- Captions/subtitles
- Playback speed
- Resume position
- Fullscreen
- Quality selection where supported
- Progress tracking
- Preview/free lectures
- Protected media URLs
- Download policy
- Video processing state

Do not claim that a browser-based system can make video impossible to download.

The goal is to make unauthorized access substantially harder through proper authorization, signed URLs and controlled delivery.

---

16. COURSE PLAYER

The course player must be a major product surface.

Include:

Left/sidebar

- Sections
- Lessons
- Completion indicators
- Lock states
- Quiz states
- Course progress

Main area

- Video/article/PDF/etc.
- Lesson title
- Description
- Resources
- Downloads where permitted
- Previous
- Next
- Complete lesson
- Notes
- Q&A
- Transcript/captions where available

Player features

- Resume playback
- Playback speed

- Fullscreen
- Captions
- Quality controls
- Keyboard shortcuts where appropriate
- Auto-save progress
- Next lesson
- Autoplay optional
- Picture-in-picture where browser-supported

---

17. STUDENT NOTES

Students must be able to create notes associated with:

course

lesson

timestamp

user

Example:

Lesson: React Hooks

Timestamp: 08:32

Note: Important explanation about useEffect dependencies.

Student can:

- Create
- Edit
- Delete
- Search
- Navigate back to timestamp

---

18. Q&A

Implement course-scoped Q&A.

Students can:

- Ask questions
- Attach permitted images/files
- Reply
- Follow discussions
- Mark useful answers

Instructor can:

- View questions
- Filter unanswered
- Respond
- Mark resolved
- Search questions

Admin can moderate.

This is separate from private support tickets.

---

19. PRIVATE SUPPORT TICKETS

Implement a separate ticket system.

Flow:

Student

↓

Instructor

↓

Admin

↓

Super Admin

Each ticket stores:

- Original question
- Course
- Student
- Instructor
- Messages
- Escalation reason
- Resolution
- Status
- Response timestamps
- SLA timestamps
- Audit history

Track:

- first response time
- resolution time
- escalation rate
- reopened tickets

---

20. COURSE COMMUNITY

Every course may have its own community.

Members:

- Enrolled students
- Assigned instructors

Features:

- Posts

- Replies
- Mentions
- Reactions
- Pinned posts
- Instructor announcements
- Moderation
- Reporting
- Search

Keep community separate from private support tickets and Q&A.

---

21. SHOPPING CART

Implement a real marketplace cart.

Features:

- Add course
- Remove course
- Save for later
- Move to wishlist
- Coupon application
- Price recalculation
- Tax calculation architecture
- Multiple courses per checkout
- Duplicate enrollment prevention
- Already-owned course detection

Cart state must be authoritative server-side for checkout.

---

22. WISHLIST

Students can:

- Add course
- Remove course
- Move course to cart
- View wishlist
- Sort/filter wishlist

Wishlist must be persistent for authenticated users.

---

23. COUPONS & PROMOTIONS

Support:

Platform coupons

Created by Admin/Super Admin.

Instructor coupons

Optional configurable capability.

Coupon rules:

- Fixed amount
- Percentage
- Course-specific
- Category-specific
- Instructor-specific
- Minimum purchase
- Maximum discount
- Start date
- Expiry date
- Usage limit
- Per-user usage limit

- First-time student
- Existing students
- Active/inactive

Never trust client-calculated discounts.

Server calculates final price.

---

24. PRICING ENGINE

Do not hard-code pricing logic inside UI components.

Create a pricing service:

Base Price

↓

Promotion

↓

Coupon

↓

Tax

↓

Final Amount

Return a server-generated pricing breakdown.

The checkout UI only displays it.

---

25. PAYMENT SYSTEM

Use a provider adapter.

Example:

PaymentProvider

├── StripeAdapter

├── PayPalAdapter

├── RazorpayAdapter

└── FutureProvider

Never couple business logic directly to Stripe/Razorpay/etc.

Payment lifecycle:

Checkout

↓

Payment Provider

↓

Webhook

↓

Signature Verification

↓

Idempotency Check

↓

Payment Record

↓

Enrollment

↓

Receipt

Never trust:

- Browser redirect
- Client payment status
- Client price
- Client enrollment status

---

26. REFUNDS

Support configurable refund policies.

The original VTI specification contains a completion-based refund concept. Keep this as a **configurable business rule**, not an irreversible architectural assumption.

Support:

- Standard refund window
- Completion-based refund
- Manual refund
- Admin approval
- Automatic refund
- Partial refund
- Refund status
- Refund reason
- Provider refund ID
- Refund audit trail

The platform must allow different deployments to use different refund policies.

---

27. SUBSCRIPTIONS

Design the architecture to support both:

Marketplace purchase

Student purchases individual course.

Subscription

Student pays monthly/yearly and receives access to a configured course collection.

Example:

SubscriptionPlan

├— name

├— price

├— billingInterval

├— currency

├— includedCourses

├— active

└— limits

Do not make subscriptions mandatory for VTI's initial deployment.

Build the architecture so the feature can be enabled later.

---

28. COURSE BUNDLES

Support:

Bundle

├— title

├— description

├— thumbnail

├— courses[]

├— price

├— discount

└— active

Students can purchase bundles where enabled.

---

29. GIFTING

Design the order system so future course gifting is possible.

Potential flow:

Buyer

↓

Course

↓

Recipient email

↓

Payment

↓

Gift record

↓

Recipient claims course

This can initially be feature-flagged.

---

30. REVIEWS

Only eligible students may review.

Requirements:

- Verified enrollment
- One active review per student/course
- 1–5 rating
- Written review
- Edit review
- Delete according to policy
- Report review
- Admin moderation
- Instructor response where enabled

Aggregate:

Average rating

Rating distribution

Review count

Do not calculate rating values independently in multiple UI components.

Use one authoritative service/query.

---

31. INSTRUCTOR PROFILE

Public instructor profile:

- Profile photo
- Name
- Professional title
- Biography
- Credentials
- Expertise
- Education
- Social links
- Website
- Number of students
- Number of courses
- Average rating
- Course list
- Reviews/feedback where appropriate

Instructor dashboard:

- Courses
- Students
- Revenue
- Performance

- Reviews
- Q&A
- Messages
- Announcements
- Coupons
- Promotions
- Course analytics
- Support tickets
- Profile

---

32. INSTRUCTOR ANALYTICS

Track:

- Enrollments
- Revenue
- Refund rate
- Course rating
- Review count
- Completion rate
- Average watch/progress
- Quiz performance
- Student engagement
- Q&A volume
- Response time
- Course conversion where data is available

Financial information must respect role permissions.

---

33. INSTRUCTOR COURSE CREATION

Create a guided course builder.

Stages:

1. Planning

2. Basic Information

3. Intended Learners

4. Curriculum

5. Content

6. Pricing

7. Promotions

8. Course Landing Page

9. Instructor

10. Captions

11. Accessibility

12. Preview

13. Validation

14. Submission

15. Moderation

16. Publishing

The workflow should visually show completion status.

A course cannot publish until required information passes validation.

Udemy's documented course-management experience includes planning, intended learners, curriculum, landing-page information, basic information, instructor profiles, pricing, promotions, course messages, students and settings; use this as a functional reference.

---

34. COURSE MODERATION

Course lifecycle:

DRAFT

↓

READY_FOR_REVIEW

↓

UNDER_REVIEW

↓

CHANGES_REQUESTED

↓

APPROVED

↓

PUBLISHED

↓

UNPUBLISHED

↓

ARCHIVED

Store moderation history.

Admin/Super Admin can:

- Approve
- Reject
- Request changes
- Unpublish
- Flag
- Archive

Instructor sees feedback.

---

35. STUDENT DASHBOARD

Create a marketplace-quality student dashboard.

Include:

- My Learning
- Continue Learning
- Recently Viewed
- Wishlist
- Cart
- Certificates
- Reviews
- Notes
- Q&A
- Announcements
- Purchase history
- Payment history
- Account settings
- Notifications
- Course progress
- Quiz performance
- Achievements
- Leaderboard
- Recommended courses

---

36. COURSE RECOMMENDATIONS

Create a recommendation architecture.

MVP:

Related category

Related topic

Same instructor

Similar price

Popular in category

Recently viewed

Later:

Collaborative filtering

Content similarity

Enrollment behavior

Completion behavior

Student interests

Recommendation logic must be isolated in a service so it can evolve.

---

37. SEARCH ARCHITECTURE

Do not design the search system around a single database query forever.

Create a search abstraction:

CourseSearchService

Initial implementation may use PostgreSQL search.

Architecture should allow future:

- Elasticsearch
- OpenSearch
- Meilisearch
- Algolia
- Typesense

without changing the public API contract.

---

38. CERTIFICATES

Certificate system must support:

- Automatic issuance
- Manual issuance if permitted
- PDF generation
- Unique certificate ID
- Verification URL
- Student snapshot
- Course snapshot
- Completion date
- Issue date
- Revocation
- Reissue policy

Public:

/verify/[certificateId]

---

39. GAMIFICATION

Optional but built into architecture:

- Badges
- Points
- Streaks
- Leaderboards
- Course completion achievements
- Quiz achievements
- Perfect score

- Quick learner
- Course champion

All awards must be idempotent.

---

40. COMMUNICATION SYSTEM

Use event-driven communication.

Events:

USER_REGISTERED

ENROLLMENT_CREATED

PAYMENT_SUCCESSFUL

COURSE_STARTED

LESSON_COMPLETED

QUIZ_PASSED

QUIZ_FAILED

COURSE_COMPLETED

CERTIFICATE_ISSUED

COURSE_PUBLISHED

COUPON_CREATED

ANNOUNCEMENT_PUBLISHED

SUPPORT_TICKET_CREATED

SUPPORT_TICKET_ESCALATED

Providers:

EmailProvider

WhatsAppProvider

NotificationProvider

Templates must be editable from Admin.

---

41. NOTIFICATION CENTER

Implement:

- In-app notifications
- Email notifications
- Optional WhatsApp notifications

Notification:

id

userId

type

title

message

link

readAt

createdAt

Support:

- Read/unread
- Mark all read
- Notification preferences
- Per-event opt-in/out where legally/business appropriate

---

42. CMS

Admin should control public content without developer intervention.

CMS-lite:

- Homepage hero
- Featured courses

- Categories
- Promotional banners
- About
- FAQ
- Contact information
- Footer links
- Announcements
- Testimonials
- Platform statistics
- Promotional sections

Do not hard-code all marketing content.

---

43. HOMEPAGE

Create a marketplace-quality homepage.

Possible sections:

Header

Hero

Search

Popular Categories

Featured Courses

Trending Courses

Top Instructors

Learning Paths

Promotional Banner

Recommended Courses

Why Choose Platform

Student Testimonials

Statistics

FAQ

CTA

Footer

The exact content should be configurable.

---

44. PUBLIC ROUTES

At minimum:

/

/courses

/courses/[slug]

/categories

/categories/[slug]

/instructors

/instructors/[slug]

/search

/about

/contact

/faq

/pricing

/announcements

/announcements/[slug]

/verify/[certificateId]

/login

/signup

/forgot-password

/reset-password/[token]

/cart

---

45. STUDENT ROUTES

/student/dashboard

/student/learning

/student/courses

/student/courses/[courseId]

/student/courses/[courseId]/learn

/student/courses/[courseId]/learn/[lessonId]

/student/courses/[courseId]/quiz/[quizId]

/student/courses/[courseId]/community

/student/courses/[courseId]/qa

/student/courses/[courseId]/notes

/student/wishlist

/student/cart

/student/orders

/student/payments

/student/certificates

/student/reviews

/student/notifications

/student/profile

/student/settings

---

46. INSTRUCTOR ROUTES

/instructor

/instructor/dashboard

/instructor/courses

/instructor/courses/new

/instructor/courses/[id]

/instructor/courses/[id]/curriculum

/instructor/courses/[id]/landing-page

/instructor/courses/[id]/pricing

/instructor/courses/[id]/promotions

/instructor/courses/[id]/students

/instructor/courses/[id]/analytics

/instructor/courses/[id]/qa

/instructor/courses/[id]/messages

/instructor/courses/[id]/announcements

/instructor/courses/[id]/reviews

/instructor/support

/instructor/profile

/instructor/settings

---

47. ADMIN ROUTES

/admin

/admin/analytics

/admin/courses

/admin/courses/new

/admin/courses/[id]

/admin/instructors

/admin/students

/admin/enrollments

/admin/orders

/admin/payments

/admin/refunds

/admin/coupons

/admin/promotions

/admin/bundles

/admin/subscriptions

/admin/reviews

/admin/qa

/admin/support

/admin/communications

/admin/announcements

/admin/certificates

/admin/gamification

/admin/categories

/admin/cms

/admin/deletion-requests

/admin/audit-logs

/admin/reports

/admin/settings

/admin/administrators

Only expose routes appropriate to the authenticated role and permissions.

---

48. DATABASE ARCHITECTURE

Use PostgreSQL + Prisma.

The schema should be modular.

Core domains:

Identity

├── User

├── Session

├── PasswordResetToken

└── Permission

People

├── InstructorProfile

└── StudentProfile

Marketplace

├── Course

├── Category

├── Subcategory

├── Topic

├── CourseInstructor

├── Wishlist

├── Cart

├── CartItem

├── Bundle

└── Recommendation

Learning

- Section/Module
- Lesson
- Quiz
- Question
- AnswerOption
- Assignment
- PracticeTest
- Enrollment
- EnrollmentAccess
- LessonProgress
- CourseProgress
- QuizAttempt
- QuizAnswer
- Note
- Certificate

Commerce

- Order
- OrderItem
- Payment
- Refund
- Invoice
- Coupon
- Promotion

├── Subscription

├── SubscriptionPlan

└── PaymentEvent

Engagement

├── Review

├── CourseQuestion

├── CourseAnswer

├── CommunityPost

├── CommunityReply

├── Notification

├── Badge

├── StudentBadge

└── Leaderboard

Support

├── SupportTicket

├── SupportMessage

├── Escalation

└── TicketAudit

Content

├── Announcement

├── FAQ

├── ContentBlock

├── SiteSetting

└── MediaAsset

Operations

├── AuditLog

├── DeletionRequest

├── ModerationAction

└── AdminPermission

Do not blindly implement every model if an existing model already satisfies the requirement.

First inspect the existing schema.

---

49. API ARCHITECTURE

API must be versioned:

/api/v1/

Domains:

/auth

/users

/courses

/categories

/instructors

/enrollments

/learning

/quizzes

/reviews

/qa

/community

/cart

/wishlist

/orders

/payments

/refunds

/coupons

/promotions

/subscriptions

/certificates

/notifications

/search

/recommendations

/support

/admin

Keep business logic outside route handlers.

Architecture:

Route

↓

Authentication

↓

Authorization

↓

Validation

↓

Service

↓

Repository/Data Access

↓

Database

---

50. SECURITY

Non-negotiable:

Never trust client-provided:

- role
- price
- discount
- payment status
- enrollment status
- progress
- quiz score
- certificate status
- course ownership
- permissions

All authoritative decisions happen server-side.

Implement:

- Secure cookies
- CSRF protection where applicable
- Rate limiting
- Input validation
- Authorization
- Resource-level access control

- Signed media URLs
- File validation
- Upload size limits
- Secure headers
- Audit logging
- Webhook signature verification
- Idempotency
- Password hashing
- Session revocation
- Brute-force protection
- Abuse protection

Follow the security requirements already established in the VTI blueprint.

---

51. AUDIT LOGGING

Audit sensitive operations:

LOGIN

LOGOUT

PASSWORD_CHANGE

USER_CREATED

USER_SUSPENDED

USER_DELETED

COURSE_CREATED

COURSE_PUBLISHED

COURSE_UNPUBLISHED

COURSE_DELETED

INSTRUCTOR_ASSIGNED

PAYMENT_CREATED

REFUND_CREATED

REFUND_COMPLETED

COUPON_CREATED

COUPON_UPDATED

REVIEW_MODERATED

SUPPORT_ESCALATED

PERMISSION_CHANGED

SETTINGS_CHANGED

Record:

- actor
- action
- entity
- entity ID
- timestamp
- IP metadata where appropriate
- request ID
- before/after metadata where appropriate

---

52. RESPONSIVE UI

The product must work on:

- Desktop
- Laptop
- Tablet
- Mobile

Do not merely shrink desktop UI.

Design intentionally for mobile.

Important:

- Course player mobile layout
- Sticky controls
- Mobile navigation
- Filter drawers
- Responsive tables
- Mobile checkout
- Mobile dashboard
- Touch-friendly controls

---

53. DESIGN SYSTEM

Use:

- Tailwind CSS
- shadcn/ui
- Lucide icons or another consistent icon library

Create a central design system.

Do NOT create random styles page-by-page.

Define:

- Colors
- Typography
- Spacing
- Radius
- Shadows
- Buttons
- Inputs

- Cards
- Dialogs
- Tables
- Tabs
- Dropdowns
- Toasts
- Badges
- Navigation
- Empty states
- Loading states
- Error states

The visual identity must be original to VTI/the eventual product.

---

54. UX QUALITY STANDARD

Every major page must have:

Loading

Empty

Error

Success

Unauthorized

Forbidden

Not Found

Mobile

Desktop

states where relevant.

Avoid:

- fake buttons
- dead links
- placeholder functionality presented as complete
- hard-coded fake analytics
- client-only security
- duplicated business logic
- inconsistent terminology
- unnecessary modals
- poor mobile UX

---

55. ANALYTICS

Platform analytics:

- Revenue
- Orders
- Refunds
- Enrollments
- Active students
- Course completion
- Average rating
- Instructor performance
- Course performance
- Conversion
- Coupon usage
- Subscription metrics
- Student engagement
- Support response times

Instructor analytics:

- Revenue
- Enrollments
- Completion
- Rating
- Reviews
- Student engagement
- Q&A
- Refunds
- Performance

Student analytics:

- Learning progress
- Quiz scores
- Completion
- Time spent
- Certificates
- Streaks

---

56. PERFORMANCE

Use:

- Server Components where appropriate
- Client Components only where interaction requires them
- Server-side data fetching where appropriate
- Pagination
- Database indexes
- Query optimization

- Image optimization
- Lazy loading
- Streaming where appropriate
- CDN
- Caching
- Background jobs for expensive operations

Do not optimize prematurely, but do not build obviously inefficient queries.

---

57. BACKGROUND JOBS

Architecture should support jobs for:

- Email
- WhatsApp
- Certificate generation
- Video processing
- Notifications
- Analytics aggregation
- Refund processing
- Recommendation generation
- Scheduled promotions
- Subscription renewals
- Cleanup
- Reporting

Use an abstraction so the implementation can evolve from simple scheduled jobs to a queue system.

---

58. OBSERVABILITY

Implement:

- Structured logs
- Request IDs
- Error tracking
- Performance monitoring
- Database monitoring
- Payment webhook monitoring
- Background job monitoring
- Email delivery monitoring
- Uptime monitoring

Development, staging and production must be separate.

---

59. TESTING STRATEGY

Use:

Unit tests

Business logic.

Integration tests

Database + service layer.

API tests

Authentication, permissions and business rules.

E2E tests

Critical journeys.

Minimum E2E flows:

Student signup

Login

Browse course

Search

Filter

Course preview

Add to wishlist

Add to cart

Apply coupon

Checkout

Payment

Enrollment

Course learning

Progress

Quiz

Certificate

Review

Q&A

Instructor login

Create course

Edit course

Create curriculum

Upload media

Submit course

Admin moderation

Publish

Admin

Create instructor

Assign instructor

Manage students

Manage courses

Process refund

Moderate review

Resolve support ticket

Super Admin

Manage admins

Approve deletion

Audit logs

Platform settings

---

60. DEFINITION OF DONE

A feature is NOT complete merely because:

- the page exists
- the button exists
- the API returns 200
- the database model exists

A system is complete only when:

Database

+

Business Logic

+

API

+

Authorization

+

UI

+

Validation

+

Error Handling

+

Mobile

+

Testing

+

Security

+

Acceptance Criteria

all work together.

---

61. DEVELOPMENT ORDER

Use dependency-first vertical development.

Phase 0

Foundation

Phase 1

Authentication + RBAC

Phase 2

Design System + Public Shell

Phase 3

Marketplace Taxonomy + Course Discovery

Phase 4

Course + Instructor Management

Phase 5

Course Authoring

Phase 6

Cart + Wishlist + Pricing

Phase 7

Orders + Payments + Coupons

Phase 8

Enrollment

Phase 9

Course Player

Phase 10

Progress + Resume + Sequential Unlocking

Phase 11

Quiz + Assignments + Practice Features

Phase 12

Student Dashboard

Phase 13

Reviews + Q&A + Notes

Phase 14

Community

Phase 15

Certificates

Phase 16

Instructor Dashboard + Analytics

Phase 17

Admin + Super Admin Operations

Phase 18

Support + Escalation

Phase 19

Communication

Phase 20

Gamification

Phase 21

Subscriptions + Bundles + Advanced Commerce

Phase 22

Search + Recommendations

Phase 23

Security Hardening

Phase 24

Performance + Observability

Phase 25

Complete QA

Phase 26

Production Launch

---

62. THE GOLDEN DEVELOPMENT RULE

DO NOT jump randomly between systems.

For every system:

1. Understand requirements

2. Inspect existing implementation

3. Identify dependencies

4. Design/update database

5. Implement services

6. Implement APIs

7. Implement authorization

8. Implement UI

9. Connect real data

10. Test positive cases

11. Test negative cases

12. Test mobile

13. Fix bugs

14. Run Definition of Done

15. Freeze system

16. Move forward

Never say:

"We'll fix this later."

If a current system has a blocking architectural defect, fix it before building dependent systems.

---

63. EXISTING CODE PROTECTION

Because substantial portions of the VTI LMS may already exist:

DO NOT:

- delete working systems without justification
- rewrite the entire application

- replace architecture simply because another approach is newer
- create duplicate models
- create duplicate APIs
- create duplicate pages
- create duplicate authentication systems

Before modifying something:

Search repository

↓

Understand existing implementation

↓

Determine dependency graph

↓

Modify minimum required files

↓

Test affected functionality

---

64. NO FAKE IMPLEMENTATION

Never create:

TODO

Coming Soon

Fake analytics

Fake payment success

Fake enrollment

Fake certificate

Fake review count

Fake student count

Fake revenue

unless explicitly marked as development fixtures.

If an external provider cannot yet be connected, create a real adapter interface and a clearly isolated development implementation.

---

65. API-FIRST / MOBILE READY

The architecture must support future:

Web

Mobile Web

React Native

iOS

Android

All business rules must live in reusable services.

Do not put important business rules exclusively inside React components.

The existing VTI blueprint explicitly calls for an API-ready architecture so future mobile applications can reuse the same backend.

---

66. REUSABILITY REQUIREMENT

The application must not contain assumptions such as:

VTI only

VTI category

VTI instructor

VTI payment

VTI-specific course type

Instead use configuration:

PlatformSettings

BrandSettings

FeatureFlags

PaymentSettings

NotificationSettings

CourseSettings

MarketplaceSettings

A future customer should be able to change:

- Logo
- Name
- Colors
- Domain
- Currency
- Payment provider
- Email provider
- Categories
- Course rules
- Certificate branding
- Communication templates
- Feature availability

without rewriting the application.

---

67. FEATURE FLAGS

Implement feature flags for advanced capabilities.

Examples:

ENABLE_CART

ENABLE_WISHLIST

ENABLE_SUBSCRIPTIONS

ENABLE_BUNDLES

ENABLE_GAMIFICATION

ENABLE_COMMUNITY

ENABLE_CODING_EXERCISES

ENABLE_ASSIGNMENTS

ENABLE_PRACTICE_TESTS

ENABLE_INSTRUCTOR_COUPONS

ENABLE_STUDENT_NOTES

ENABLE_QA

ENABLE_WHATSAPP

ENABLE_RECOMMENDATIONS

This allows the same product to be sold to different clients.

---

68. MULTI-TENANT READINESS

Do not necessarily implement full multi-tenancy immediately.

However, design domain boundaries so future support is possible.

Potential future structure:

Platform

├— Organization

│ ├— Admins

│ ├— Instructors

│ ├— Students

│ ├— Courses

│ └— Settings

Do not allow future tenancy requirements to contaminate the initial implementation unnecessarily.

Prepare the architecture intelligently.

---

69. BUSINESS MODEL FLEXIBILITY

The platform must support multiple deployment models.

Model A — Private LMS

One organization owns all courses.

Model B — Marketplace

Multiple instructors publish courses.

Model C — Academy

Organization + instructors + enrolled students.

Model D — Subscription platform

Students subscribe to course collections.

Model E — Hybrid

Individual purchases + subscriptions.

The core architecture should support all five.

---

70. DATA OWNERSHIP

Clearly distinguish:

Platform-owned data

Organization-owned data

Instructor-owned content

Student-owned data

Financial records

Historical/audit records

Never allow deletion of historical financial/audit information merely because a user requests account deletion.

Use archival/anonymization policies where appropriate.

---

71. SEO

Public marketplace pages must be SEO-friendly.

Implement:

- Metadata
- Open Graph
- Twitter/X metadata
- Canonical URLs
- Structured data where appropriate
- Sitemap
- Robots
- Course schema where appropriate
- Instructor profile metadata
- Category pages
- Clean URLs

Do not expose private student content to search engines.

---

72. ACCESSIBILITY

Target WCAG-conscious implementation.

Support:

- Keyboard navigation
- Visible focus
- Semantic HTML
- Labels
- Accessible dialogs

- Screen-reader-friendly controls
- Contrast
- Captions
- Accessible forms
- Error messages
- Reduced-motion consideration

---

73. INTERNATIONALIZATION READINESS

Do not hard-code English strings everywhere.

Prepare for:

English

Urdu

Arabic

Spanish

etc.

Architecture should support:

- Translation keys
- Locale
- Currency
- Date/time formatting
- RTL

RTL support should be considered because VTI may eventually serve Urdu/Arabic users.

---

74. CURRENCY

Do not hard-code USD.

Use:

Currency

ExchangeRate

Money

Price

Financial calculations must use integer minor units where appropriate.

Support future:

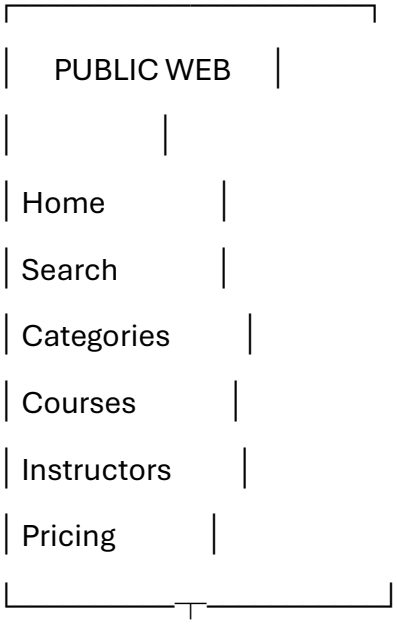
- USD
- PKR
- EUR
- GBP
- AED
- SAR

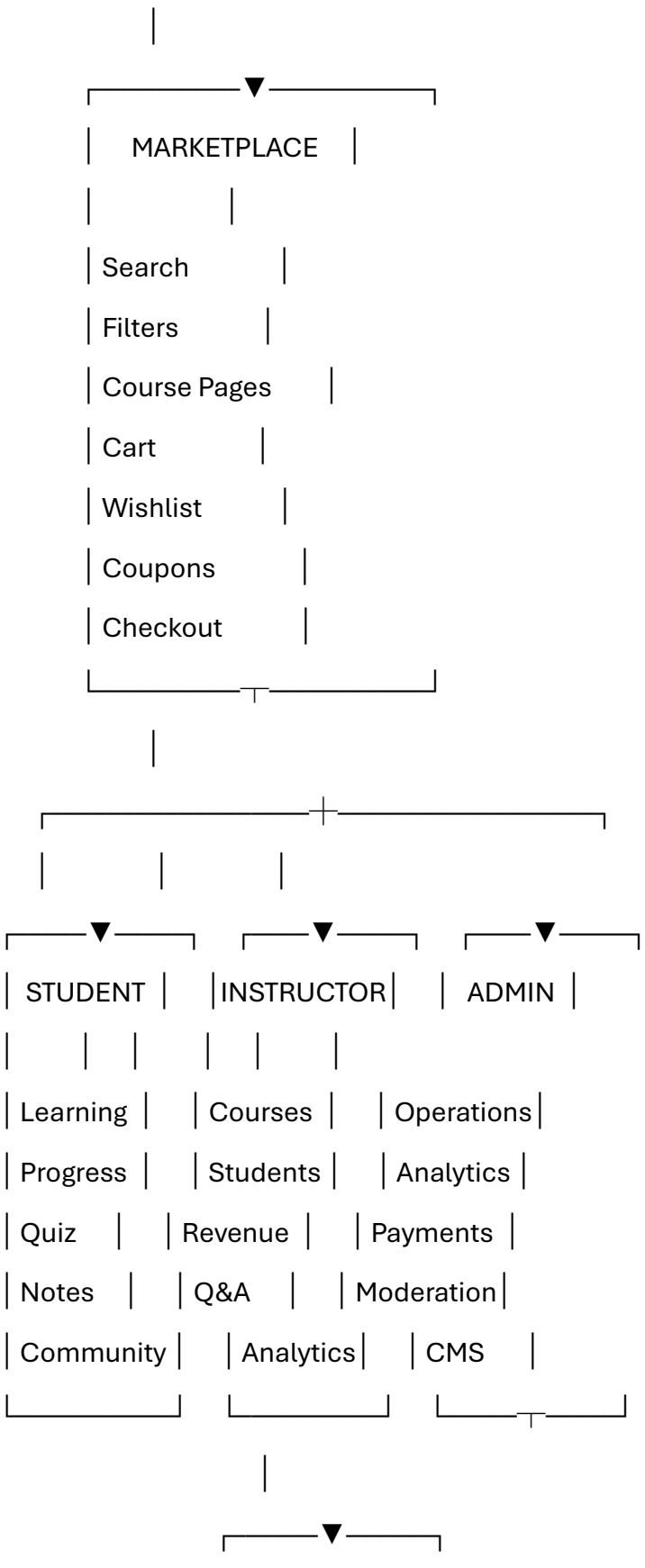
without rewriting pricing logic.

---

75. FINAL PRODUCT STRUCTURE

The final platform should conceptually look like:




MARKETPLACE
Search
Filters
Course Pages
Cart
Wishlist
Coupons
Checkout
STUDENT
Learning
Progress
Quiz
Notes
Community
INSTRUCTOR
Courses
Students
Revenue
Q&A
Analytics
ADMIN
Operations
Analytics
Payments
Moderation
CMS



---

76. FINAL QUALITY BAR

Before declaring the project finished, ask:

Marketplace

- Can a visitor discover courses?
- Can they search?
- Can they filter?
- Can they compare course information?
- Can they preview?
- Can they wishlist?
- Can they add multiple courses to cart?
- Can they use coupons?
- Can they purchase?
- Can they review?

Learning

- Can students learn?
- Can they resume?
- Can they take quizzes?

- Can they see resources?
- Can they take notes?
- Can they participate in Q&A?
- Can they communicate with instructors?
- Can they receive certificates?

Instructor

- Can instructors create courses?
- Can they build curriculum?
- Can they upload content?
- Can they manage students?
- Can they answer Q&A?
- Can they see analytics?
- Can they communicate?
- Can they manage promotions where permitted?

Administration

- Can Admin manage operations?
- Can Super Admin control the platform?
- Are permissions granular?
- Are destructive operations protected?
- Are deletion requests auditable?
- Can financial activity be tracked?
- Can courses be moderated?

Engineering

- Is authorization server-side?
- Are payments webhook verified?
- Are operations idempotent?

- Are private media protected?
- Are tests present?
- Does mobile work?
- Is the API reusable?
- Is the architecture maintainable?

If any major answer is "no", the product is not finished.

---

77. YOUR OPERATING PRINCIPLE

Always think:

Build the platform, not the page.

A page is merely a view over the underlying system.

For example:

Course Page

is not merely:

React component

It depends on:

Course

Category

Instructor

Pricing

Promotion

Reviews

Enrollment

Preview

Curriculum

Recommendations

SEO

Analytics

Authorization

Likewise:

Checkout

is not merely:

Payment button

It depends on:

Cart

Pricing

Coupons

Tax

Order

Payment

Webhook

Idempotency

Enrollment

Invoice

Notification

Audit

Use **system thinking and dependency-first architecture** throughout the project.

---

78. FINAL INSTRUCTION TO GEMINI / ANTIGRAVITY

You are allowed to browse the live public Udemy website and official documentation to understand current product functionality.

Before implementing any major marketplace subsystem:

1. Inspect the current VTI repository.
2. Inspect the existing implementation.
3. Research the corresponding current Udemy workflow.
4. Extract the functional requirements.
5. Adapt those requirements to VTI's architecture.
6. Create an original implementation.
7. Do not copy Udemy proprietary code, assets, branding or private functionality.
8. Preserve existing working VTI functionality.
9. Implement the complete vertical slice.
10. Test it.
11. Fix it.
12. Only then move to the next subsystem.

When the live Udemy website changes, use the current publicly observable product as the reference rather than assuming that older documentation represents the current UI.

The goal is:

A commercially reusable, premium, Udemy-class learning marketplace and LMS engine — with VTI-specific requirements implemented as configuration/business rules rather than hard-coded assumptions.

Do not optimize for the smallest amount of code.

Optimize for:

product completeness + maintainability + security + reusability + UX quality + extensibility.